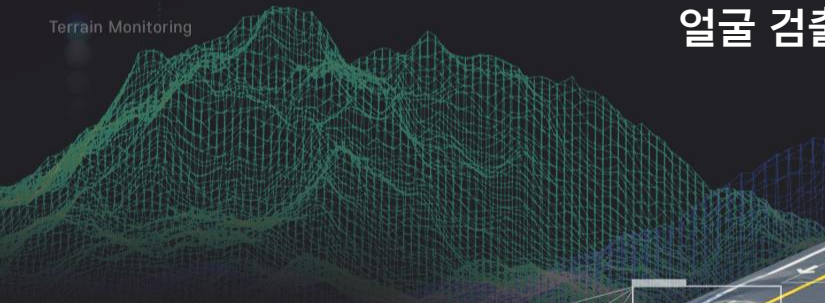


Image Recognition



Terrain Monitoring



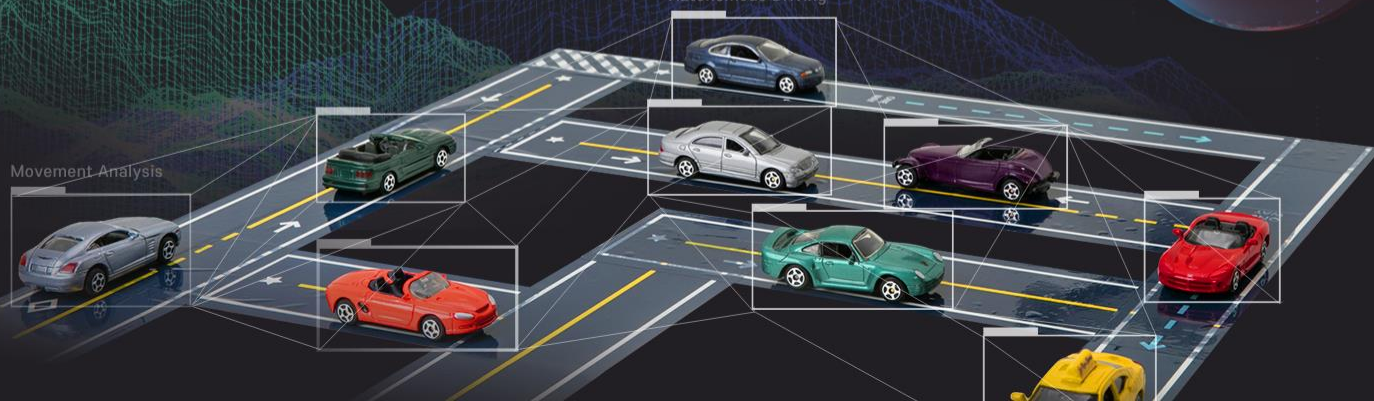
강원혁신플랫폼

컴퓨터 비전

얼굴 검출(haar cascade) 비디오

Autonomous Driving

Movement Analysis



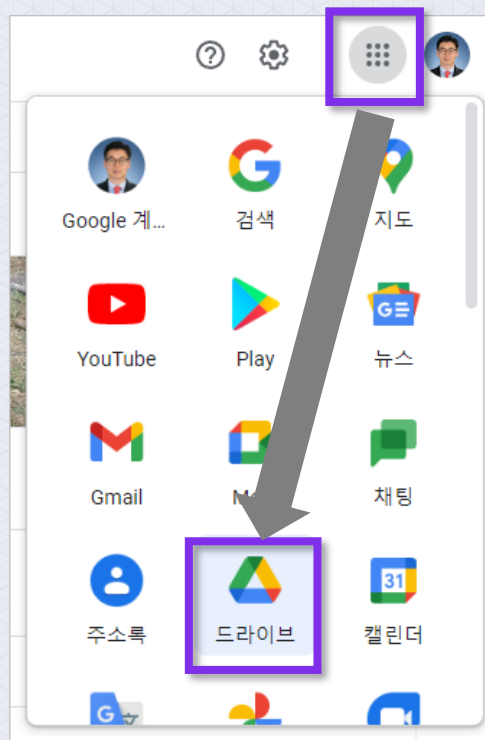
■ 비디오에서 얼굴 검출(Video Face Detection)

- Haar Cascade(하르 특징 분류기) 객체 감지 방식은 사전에 학습된 haarcascade module을 사용하여 비디오에서 얼굴과 눈 등을 감지함

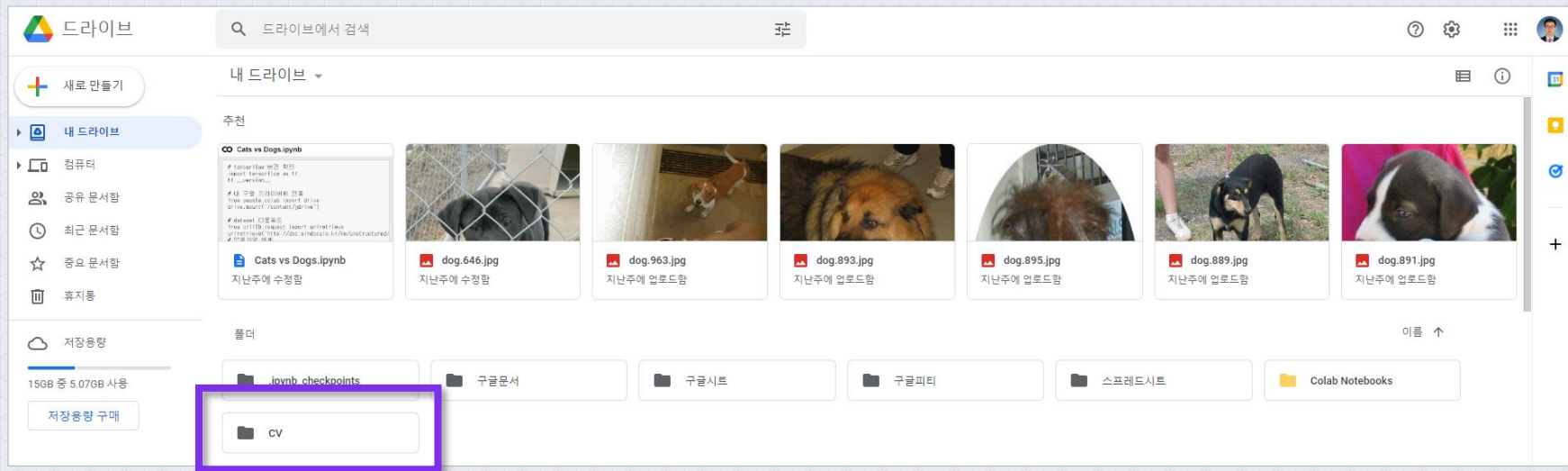
비디오에서 정면 얼굴 검출한 결과



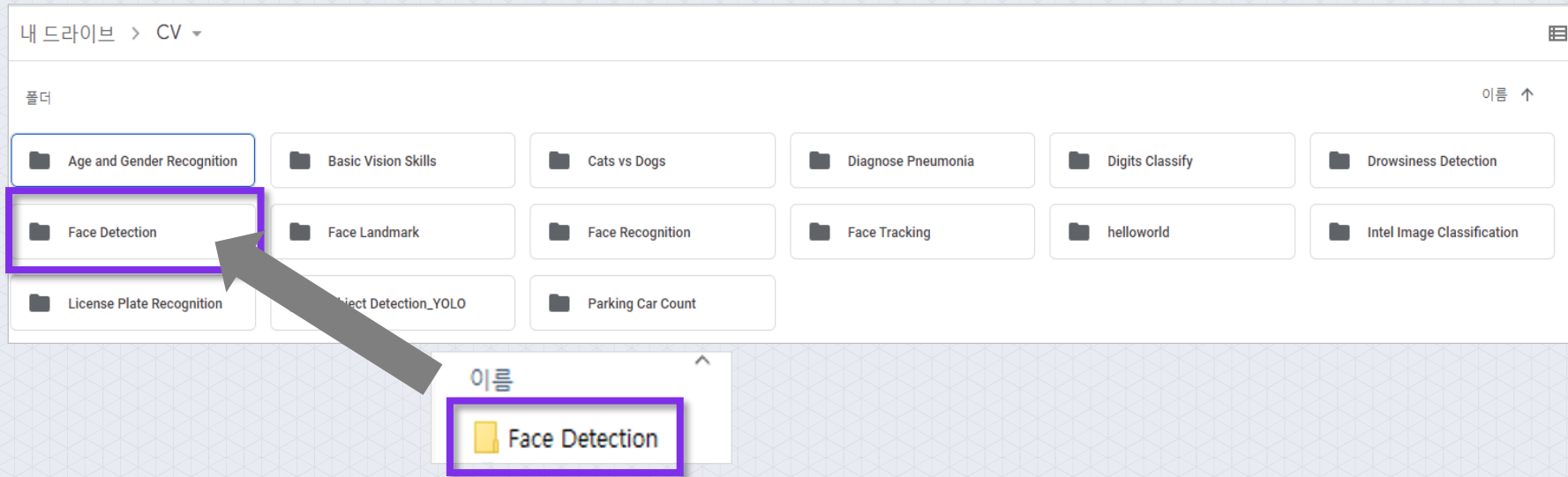
- 먼저, 구글 Colab을 이용해 보자
- 구글 크롬 브라우저에서 아래 그림과 같이 **구글 드라이브**를 클릭



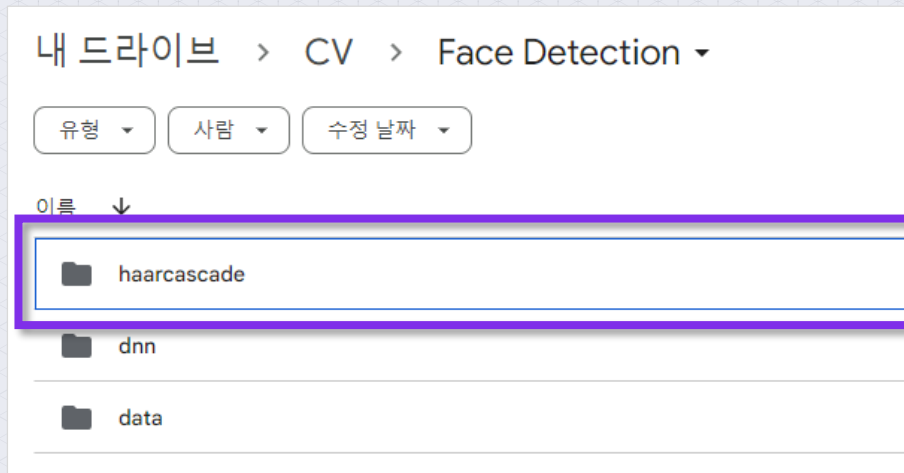
구글 드라이브에서 CV 폴더를 클릭



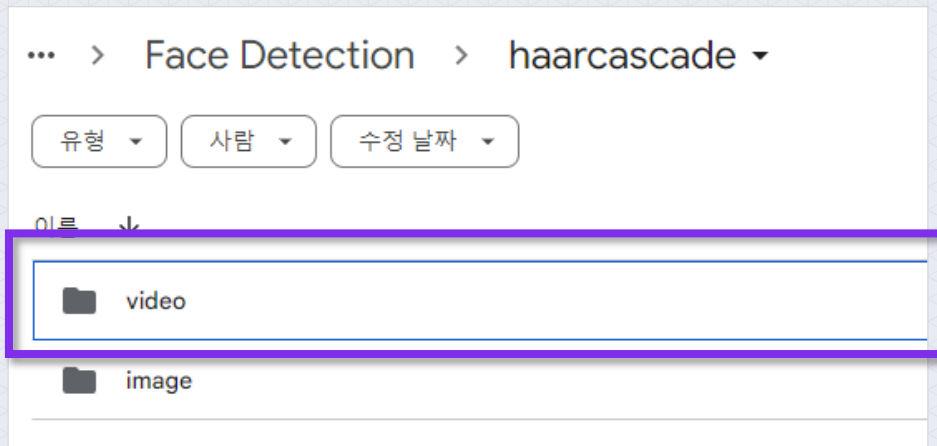
- 구글 드라이브 > CV > Face Detection 폴더를 클릭
- 강의 실습 자료를 다운로드하여 [Face Detection] 폴더를 구글 드라이브 CV 폴더 아래에 복사(업로드)



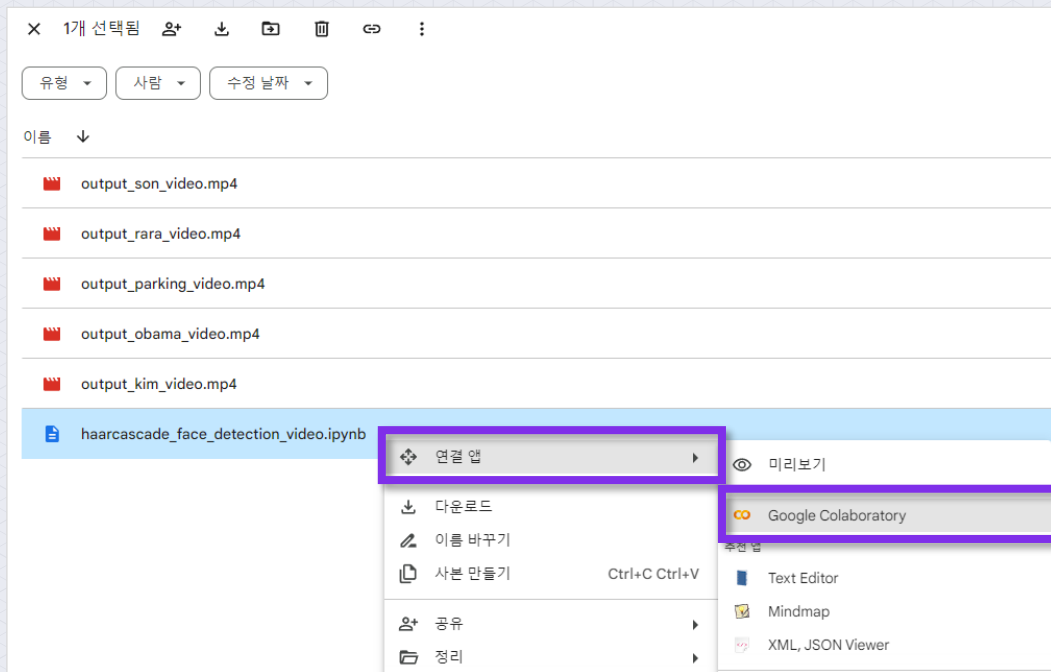
- ➡ 구글 드라이브 > CV > Face Detection > haarcascade 폴더 클릭
- ➡ Face Detection > haarcascade 폴더 클릭



- ➡ 구글 드라이브 > CV > Face Detection > haarcascade > video 폴더 클릭
- ➡ Face Detection > haarcascade > video 폴더 클릭



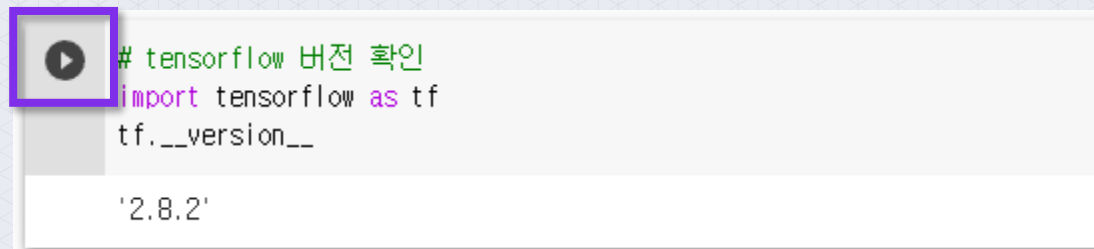
- 🕒 video 폴더에서 [haarcascade_face_detection_video.ipynb](#)를 선택 후 오른쪽 마우스를 클릭하여 “연결 앱 > Google Colaboratory” 클릭



🔄 구글 Colab 노트북 화면이 보이면 실습 준비가 완료된 것



- 구글 Colab 노트북 화면에서 실행 버튼을 클릭



```
# tensorflow 버전 확인
import tensorflow as tf
tf.__version__

'2.8.2'
```

- 실행 결과 tensorflow 버전이 '2.8.2'인 것을 알 수 있음

- 내 구글 드라이브에 연동
- 구글 Colab 노트북 화면에서 실행 버튼 클릭



- [Google Drive에 연결] > [계정 선택] > [구글 드라이브 접근 허용] 메뉴를 선택하면 gdrive에 연동
- gdrive에 연동되면 구글 드라이브에 저장된 데이터를 활용할 수 있음

구글 Colab 화면에서 내 구글 드라이브 연동

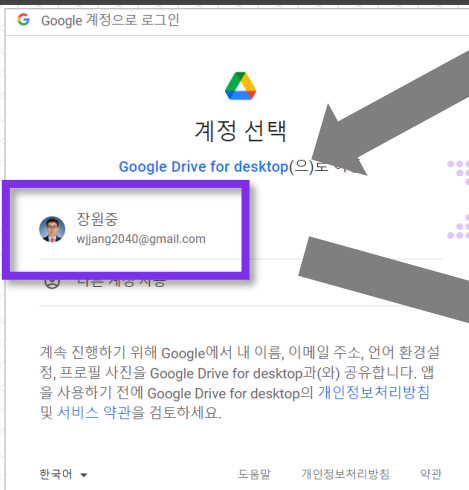
- [Google Drive에 연결] > [계정 선택] > [구글 드라이버 접근 허용]
- 아래 화면을 참고하여 **gdrive**에 연동

노트북에서 Google Drive 파일에 액세스하도록 허용하시겠습니까?

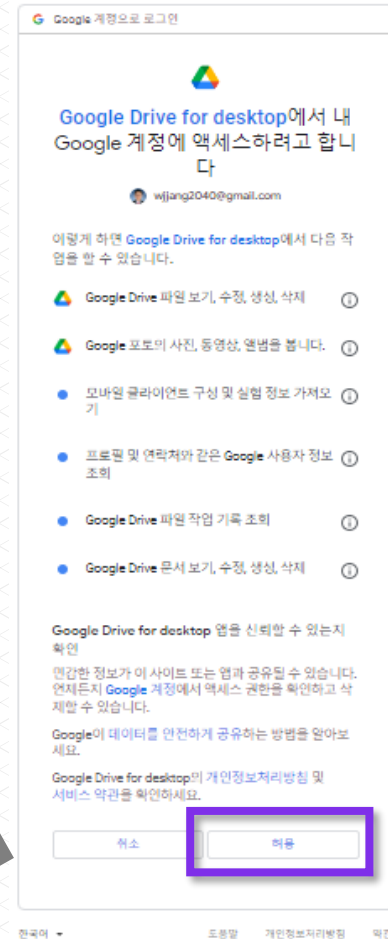
이 노트북에서 Google Drive 파일에 대한 액세스를 요청합니다. Google Drive에 대한 액세스 권한을 부여하면 노트북에서 실행되는 코드가 Google Drive의 파일을 수정할 수 있게 됩니다. 이 액세스를 허용하기 전에 노트북 코드를 검토하시기 바랍니다.

아니오

Google Drive에 연결



구글 드라이버 접근 허용



- 필요한 패키지와 모듈 불러오기
- 구글 Colab 노트북 화면에서 실행 버튼 클릭



필요한 패키지와 모듈 불러오기

```
import cv2
import numpy as np
import time
import io
import base64
from IPython.display import HTML
```

- OpenCV 버전을 확인
- 구글 Colab 노트북 화면에서 실행 버튼을 클릭

```
1  # OpenCV 버전을 확인
2
3  print("OpenCV version:")
4  print(cv2.__version__)
```

```
OpenCV version:
4.7.0
```

🔄 비디오 이미지에서 얼굴 검출(Face detection)하기 전에 원본 비디오 화면 출력

```
1 # Detection 하기 전에 원본 동영상 Display
2
3 video = io.open('gdrive/My Drive/CV/Face Detection/data/video/son.mp4', 'r+b').read()
4 encoded = base64.b64encode(video)
5 HTML(data='''<video width="50%" controls>
6 | | | | | <source src="data:video/mp4;base64,{0}" type="video/mp4"/>
7 | | | | | </video>'''.format(encoded.decode('ascii')))
```

🔄 “손흥민” 선수의 골 세리머니 장면



🔄 비디오 이미지에서 얼굴 검출(Face detection)하기 전에 원본 비디오 화면 출력

```
1 # Detection 하기 전에 원본 동영상 Display
2
3 video2 = io.open('gdrive/My Drive/CV/Face Detection/data/video/kim.mp4', 'r+b').read()
4 encoded = base64.b64encode(video2)
5 HTML(data='''<video width="50%" controls>
6 | | | | | <source src="data:video/mp4;base64,{0}" type="video/mp4"/>
7 | | | | | </video>'''.format(encoded.decode('ascii')))
```



🔄 “김연경” 선수의 득점 세리머니 장면

비디오 이미지에서 얼굴 검출(Face detection)하기 전에 원본 비디오 화면 출력

```
1 # Detection 하기 전에 원본 동영상 Display
2
3 video3 = io.open('gdrive/My Drive/CV/Face Detection/data/video/obama.mp4', 'r+b').read()
4 encoded = base64.b64encode(video3)
5 HTML(data='''<video width="50%" controls>
6         <source src="data:video/mp4;base64,{0}" type="video/mp4"/>
7         </video>'''.format(encoded.decode('ascii')))
```

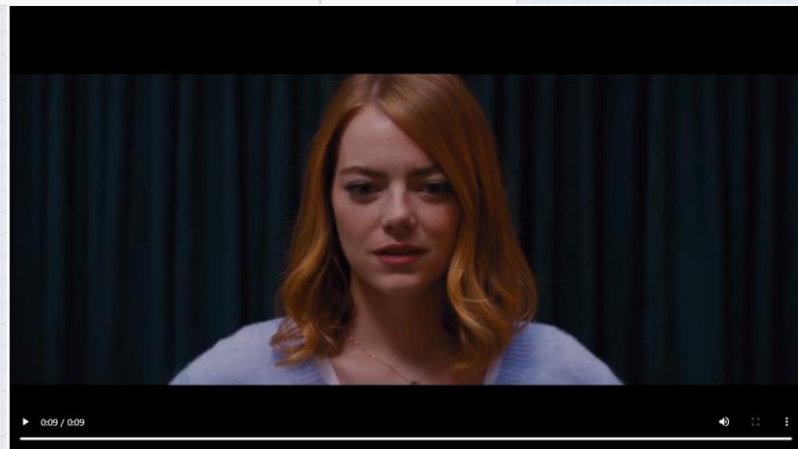


“버락 오바마” 미국 전 대통령의 연설 장면

🔄 비디오 이미지에서 얼굴 검출(Face detection)하기 전에 원본 비디오 화면 출력

```
1 # Detection 하기 전에 원본 동영상 Display
2
3 video4 = io.open('gdrive/My Drive/CV/Face Detection/data/video/rara.mp4', 'r+b').read()
4 encoded = base64.b64encode(video4)
5 HTML(data='''<video width="50%" controls>
6 | | | | | <source src="data:video/mp4;base64,{0}" type="video/mp4"/>
7 | | | | | </video>'''.format(encoded.decode('ascii')))
```

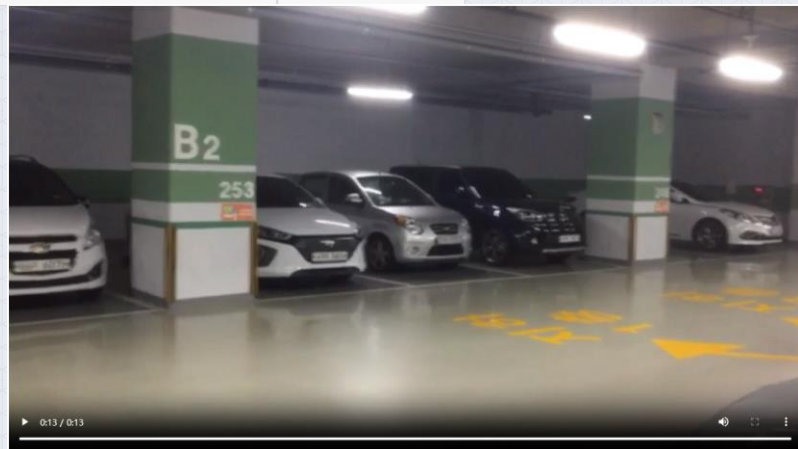
🔄 “라라” 씨의 연극 장면



🔄 비디오 이미지에서 얼굴 검출(Face detection)하기 전에 원본 비디오 화면 출력

```
1 # Detection 하기 전에 원본 동영상 Display
2
3 video5 = io.open('gdrive/My Drive/CV/Face Detection/data/video/parking.mp4', 'r+b').read()
4 encoded = base64.b64encode(video5)
5 HTML(data='''<video width="50%" controls>
6 | | | | | <source src="data:video/mp4;base64,{0}" type="video/mp4"/>
7 | | | | | </video>'''.format(encoded.decode('ascii')))
```

🔄 지하 주차장 녹화 영상



- 특정값이 미리 정의되어 있는 haarcascade module(xml 파일) 위치를 정의함
- cv2.CascadeClassifier() 함수를 통해 정면 얼굴, 측면 얼굴, 눈, 상반신, 웃음, 자동차 번호판 인스턴스를 생성

haarcascade_frontalface_alt.xml	전면 얼굴 기본 검출기 xml 파일
haarcascade_profileface.xml	측면 얼굴을 인식하는 기본 검출기 xml 파일
haarcascade_eye_tree_eyeglasses.xml	눈을 인식하는 기본 검출기 xml 파일
haarcascade_upperbody.xml	상반신을 인식하는 기본 검출기 xml 파일
haarcascade_smile.xml	웃음을 인식하는 기본 검출기 xml 파일
haarcascade_Russian_plate_number.xml	자동차 번호판을 인식하는 기본 검출기 xml 파일

```
1 # 특성이 이미 정의되어 있는 haarcascade feature 파일 xml 위치를 정의
2
3 # 정면 얼굴을 인식하는 feature 파일
4 front_face_cascade_name = 'gdrive/My Drive/CV/Face Detection/data/haarcascades/haarcascade_frontalface_alt.xml'
5
6 # 측면 얼굴을 인식하는 feature 파일
7 side_face_cascade_name = 'gdrive/My Drive/CV/Face Detection/data/haarcascades/haarcascade_profileface.xml'
8
9 # 눈을 인식하는 feature 파일
10 eyes_cascade_name = 'gdrive/My Drive/CV/Face Detection/data/haarcascades/haarcascade_eye_tree_eyeglasses.xml'
11
12 # 사람의 상반신을 인식하는 feature 파일
13 upper_body_cascade_name = 'gdrive/My Drive/CV/Face Detection/data/haarcascades/haarcascade_upperbody.xml'
14
15 # 사람의 웃음 인식하는 feature 파일
16 smile_cascade_name = 'gdrive/My Drive/CV/Face Detection/data/haarcascades/haarcascade_smile.xml'
17
18 # 자동차 번호판을 인식하는 feature 파일
19 plate_number_cascade_name = 'gdrive/My Drive/CV/Face Detection/data/haarcascades/haarcascade_russian_plate_number.xml'
20
21 # front_face_cascade 인스턴스를 생성한다.
22 front_face_cascade = cv2.CascadeClassifier()
23
24 # side_face_cascade 인스턴스를 생성한다.
25 side_face_cascade = cv2.CascadeClassifier()
26
27 # eyes_cascade 인스턴스를 생성한다.
28 eyes_cascade = cv2.CascadeClassifier()
29
30 # upperbody_cascade 인스턴스를 생성한다.
31 upperbody_cascade = cv2.CascadeClassifier()
32
33 # smile_cascade 인스턴스를 생성한다.
34 smile_cascade = cv2.CascadeClassifier()
35
36 # plate_number_cascade 인스턴스를 생성한다.
37 plate_number_cascade = cv2.CascadeClassifier()
```

➡ load() 함수로 Haar Cascade 특성값이 미리 정의되어 있는 xml 파일을 불러옴

```
1 # cascades 파일을 불러온다.
2
3 if not front_face_cascade.load(cv2.samples.findFile(front_face_cascade_name)): # 사람의 정면 얼굴
4     print('--(!)Error loading face cascade')
5     exit(0)
6
7 if not side_face_cascade.load(cv2.samples.findFile(side_face_cascade_name)): # 사람의 측면 얼굴
8     print('--(!)Error loading face cascade')
9     exit(0)
10
11 if not eyes_cascade.load(cv2.samples.findFile(eyes_cascade_name)): # 사람의 눈
12     print('--(!)Error loading eyes cascade')
13     exit(0)
14
15 if not upperbody_cascade.load(cv2.samples.findFile(upper_body_cascade_name)): # 사람의 상반신
16     print('--(!)Error loading eyes cascade')
17     exit(0)
18
19 if not smile_cascade.load(cv2.samples.findFile(smile_cascade_name)): # 웃음
20     print('--(!)Error loading eyes cascade')
21     exit(0)
22
23 if not plate_number_cascade.load(cv2.samples.findFile(plate_number_cascade_name)): # 자동차 번호판
24     print('--(!)Error loading eyes cascade')
25     exit(0)
```


비디오 이미지에서 얼굴 검출하는 함수 정의

- 비디오 이미지에서 얼굴 검출(Face detection)하고 정의된 파일로 저장하는 함수를 정의
- detectAndDisplay() 함수는 얼굴을 검출하여 파일(output_name)로 저장

```
def detectAndDisplay(frame):
    # GRAY 이미지로 변환
    frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    frame_gray = cv2.equalizeHist(frame_gray)

    #-- Detect faces
    faces = front_face_cascade.detectMultiScale(frame_gray, scaleFactor=1.1, minNeighbors=4)
    for (x,y,w,h) in faces:
        center = (x + w//2, y + h//2)
        frame = cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 4) # detection 된 얼굴에 사각형을 그린다.
        faceROI = frame_gray[y:y+h,x:x+w] # frame_gray에서 방금 그린 얼굴 사각형 영역만 가져온다.(faceROI)

        # faceROI 에서 eyes 를 detection 한다.
        eyes = eyes_cascade.detectMultiScale(faceROI, scaleFactor=1.1, minNeighbors=4)
        for (x2,y2,w2,h2) in eyes:
            eye_center = (x + x2 + w2//2, y + y2 + h2//2)
            radius = int(round((w2 + h2)*0.25)) # 반지름 계산
            frame = cv2.circle(frame, eye_center, radius, (255, 0, 0 ), 4) # detection 된 눈에 원을 그린다.

        # faceROI 에서 웃음을 detection 한다.
        smiles = smile_cascade.detectMultiScale(faceROI, scaleFactor=1.1, minNeighbors=4)
        for (x3,y3,w3,h3) in smiles:
            smile_center = (x + x3 + w3//2, y + y3 + h3//2)
            radius = int(round((w3 + h3)*0.25)) # 반지름 계산
            frame = cv2.circle(frame, smile_center, radius, (0, 0, 255 ), 2) # detection 된 눈에 원을 그린다.

    # video 를 disk 에 output 하기 위해 writer 를 초기화한다.
    global writer
    if writer is None and output_name is not None:
        fourcc = cv2.VideoWriter_fourcc(*"MJPG") # 코덱 정의
        writer = cv2.VideoWriter(output_name, fourcc, 30,
                                (frame.shape[1], frame.shape[0]), True)

    # disk 에 frame 을 write 합니다.
    if writer is not None:
        writer.write(frame)
```

여기서는 원본 동영상에서
정면 얼굴, 눈, 웃음을 인식함

- 원본 동영상을 프레임 단위로 읽어서 정면 얼굴, 눈, 웃음을 인식하여 저장
- “son.mp4” 원본 동영상 파일을 이용
- 얼굴(정면 얼굴, 눈, 웃음) 검출 및 저장에는 detectAndDisplay() 함수 이용
- 얼굴 검출된 출력은 “output_son_video.mp4” 파일에 저장됨

```
1  # son.mp4 원본 동영상에서 video stream을 읽어온다.
2
3  file_name1 = 'gdrive/My Drive/CV/Face Detection/data/video/son.mp4' # 원본 동영상
4  output_name = 'output_son_video.mp4' # detection 된 output 동영상
5
6  cap = cv2.VideoCapture(file_name1) # 원본 동영상
7
8  writer = None
9  if not cap.isOpened:
10     print('--(!)Error opening video capture')
11     exit(0)
12
13  while True:
14     ret, frame = cap.read()
15     if frame is None:
16         # close the video file pointers
17         cap.release()
18         # close the writer point
19         writer.release()
20         print('--(!) No captured frame -- Break!')
21         break
22     detectAndDisplay(frame) # frame 을 보낸다.
```

- VM에 저장된 “output_son_video.mp4” 파일이 생성된 것을 확인

아래와 같이 스크립트 실행

```
1 # 구글 드라이브에 output_son_video.mp4 파일 확인
2
3 !!s
```

gdrive output_son_video.mp4 sample_data

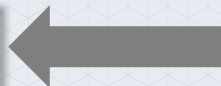
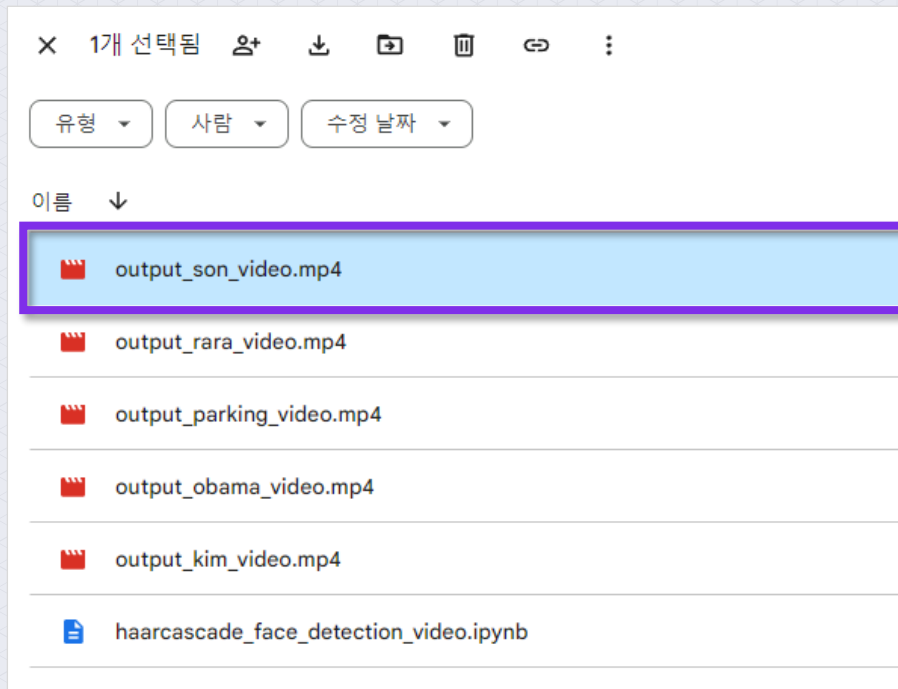
- VM에 저장된 “output_son_video.mp4” 파일을 내 구글 드라이브로 이동시킴

아래와 같이 스크립트 실행

```
1 # 동영상에서 얼굴 검출을 저장하고, 그 파일을 내 구글 드라이브로 이동시킴
2
3 !mv output_son_video.mp4 /content/gdrive/My Drive/CV/Face Detection/haarcascade/video/
```

■ 내 구글 드라이브에 “output_son_video.mp4” 파일을 실행해 보자

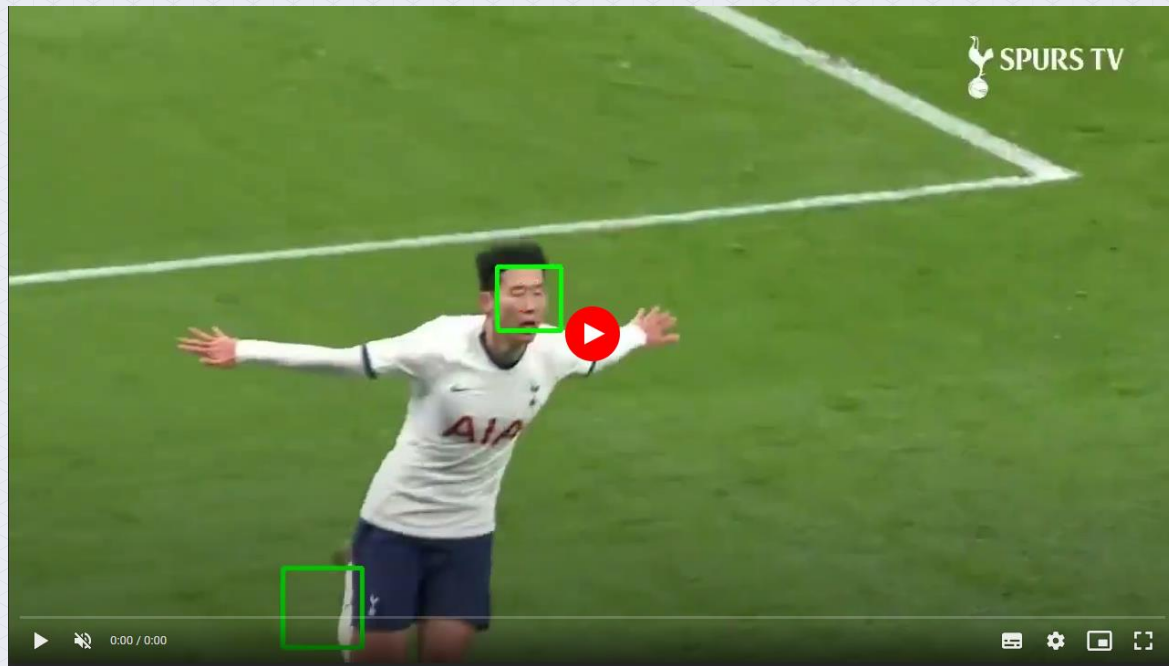
🔄 CV/Face Detection/haarcascade/video/output_son_video.mp4



더블 클릭하여 실행

- 영상을 확인해 보고 **파라미터(parameter)**를 **조정하면서 정확도를 개선해 보자**

🔄 CV/Face Detection/haarcascade/video/output_son_video.mp4



- 원본 동영상을 프레임 단위로 읽어서 정면 얼굴, 눈, 웃음을 인식하여 저장
- 여기서는 “kim.mp4” 원본 동영상 파일을 이용
- 얼굴(정면 얼굴, 눈, 웃음) 검출 및 저장에는 detectAndDisplay() 함수 이용
- 얼굴 검출된 출력은 “output_kim_video.mp4” 파일에 저장됨

```
1  # kim.mp4 원본 동영상에서 video stream을 읽어온다.
2
3  file_name2 = 'gdrive/My Drive/CV/Face Detection/data/video/kim.mp4' # 원본 동영상
4  output_name = 'output_kim_video.mp4' # detection 된 output 동영상
5
6  cap = cv2.VideoCapture(file_name2) # 원본 동영상
7
8  writer = None
9  if not cap.isOpened:
10     print('--(!)Error opening video capture')
11     exit(0)
12
13  while True:
14     ret, frame = cap.read()
15     if frame is None:
16         # close the video file pointers
17         cap.release()
18         # close the writer point
19         writer.release()
20         print('--(!) No captured frame -- Break!')
21         break
22     detectAndDisplay(frame) # frame 을 보낸다.
```

- VM에 저장된 “output_kim_video.mp4” 파일이 생성된 것을 확인

아래와 같이 스크립트 실행

```
1 # 구글 드라이브에 output_kim_video.mp4 파일 확인
2
3 !ls
```

gdrive output_kim_video.mp4 sample_data

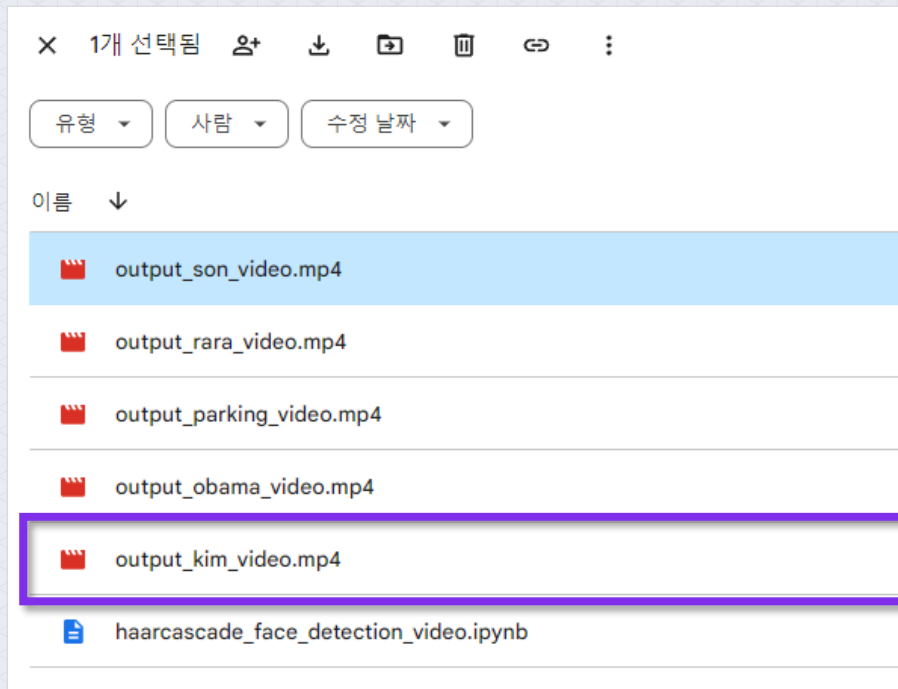
- VM에 저장된 “output_kim_video.mp4” 파일을 내 구글 드라이브로 이동시킴

아래와 같이 스크립트 실행

```
1 # 동영상에서 얼굴 검출을 저장하고, 그 파일을 내 구글 드라이브로 이동시킴
2
3 !mv output_kim_video.mp4 /content/gdrive/My Drive/CV/Face Detection/haarcascade/video/
```


■ 내 구글 드라이브에 “output_kim_video.mp4” 파일을 실행해 보자

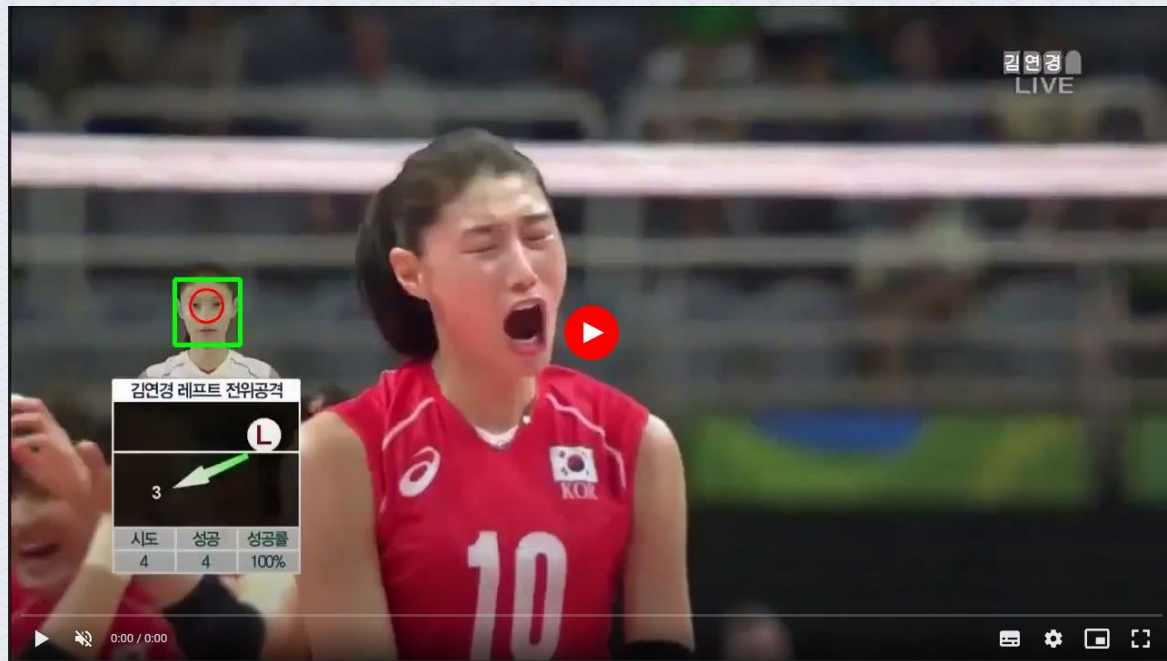
🔄 CV/Face Detection/haarcascade/video/output_kim_video.mp4



더블 클릭하여 실행

■ 영상을 확인해 보고 **파라미터(parameter)**를 **조정하면서 정확도를 개선해 보자**

🔄 CV/Face Detection/haarcascade/video/output_kim_video.mp4



- 원본 동영상을 프레임 단위로 읽어서 정면 얼굴, 눈, 웃음을 인식하여 저장
- 여기서는 “obama.mp4” 원본 동영상 파일을 이용
- 얼굴(정면 얼굴, 눈, 웃음) 검출 및 저장에는 detectAndDisplay() 함수 이용
- 얼굴 검출된 출력은 “output_obama_video.mp4” 파일에 저장됨

```
1  # obama.mp4 원본 동영상에서 video stream을 읽어온다.
2
3  file_name3 = 'gdrive/My Drive/CV/Face Detection/data/video/obama.mp4' # 원본 동영상
4  output_name = 'output_obama_video.mp4' # detection 된 output 동영상
5
6  cap = cv2.VideoCapture(file_name3) # 원본 동영상
7
8  writer = None
9  if not cap.isOpened:
10     print('--(!)Error opening video capture')
11     exit(0)
12
13  while True:
14     ret, frame = cap.read()
15     if frame is None:
16         # close the video file pointers
17         cap.release()
18         # close the writer point
19         writer.release()
20         print('--(!) No captured frame -- Break!')
21         break
22     detectAndDisplay(frame) # frame 을 보낸다.
```

- VM에 저장된 “output_obama_video.mp4” 파일이 생성된 것을 확인

아래와 같이 스크립트 실행

```
1 # 구글 드라이브에 output_obama_video.mp4 파일 확인
2
3 !!s
```

gdrive output_obama_video.mp4 sample_data

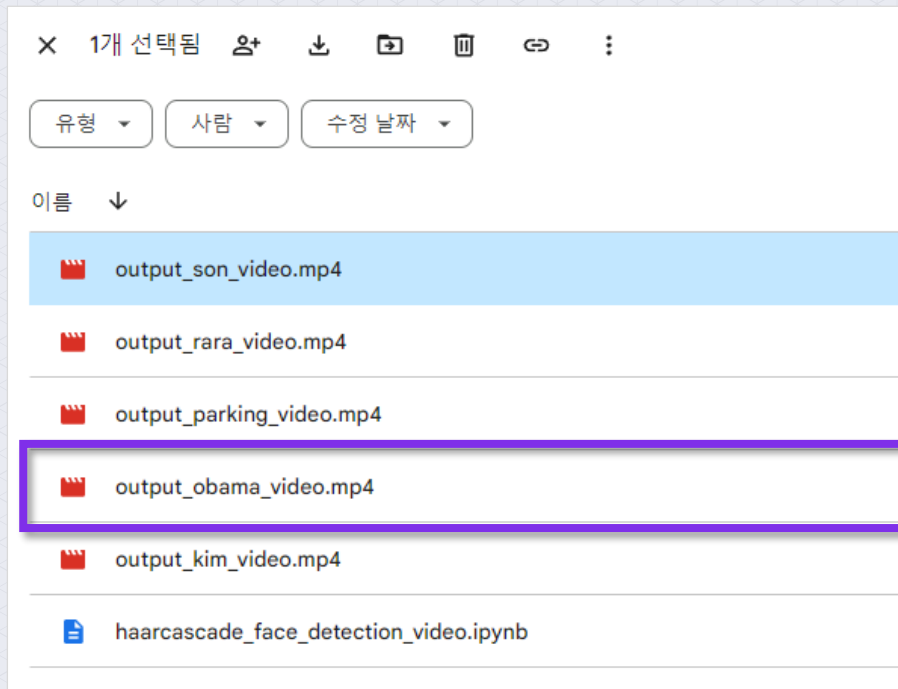
- VM에 저장된 “output_obama_video.mp4” 파일을 내 구글 드라이브로 이동시킴

아래와 같이 스크립트 실행

```
1 # 동영상에서 얼굴 검출을 저장하고, 그 파일을 내 구글 드라이브로 이동시킴
2
3 !mv output_obama_video.mp4 /content/gdrive/My Drive/CV/Face Detection/haarcascade/video/
```

■ 내 구글 드라이브에 “output_obama_video.mp4” 파일을 실행해 보자

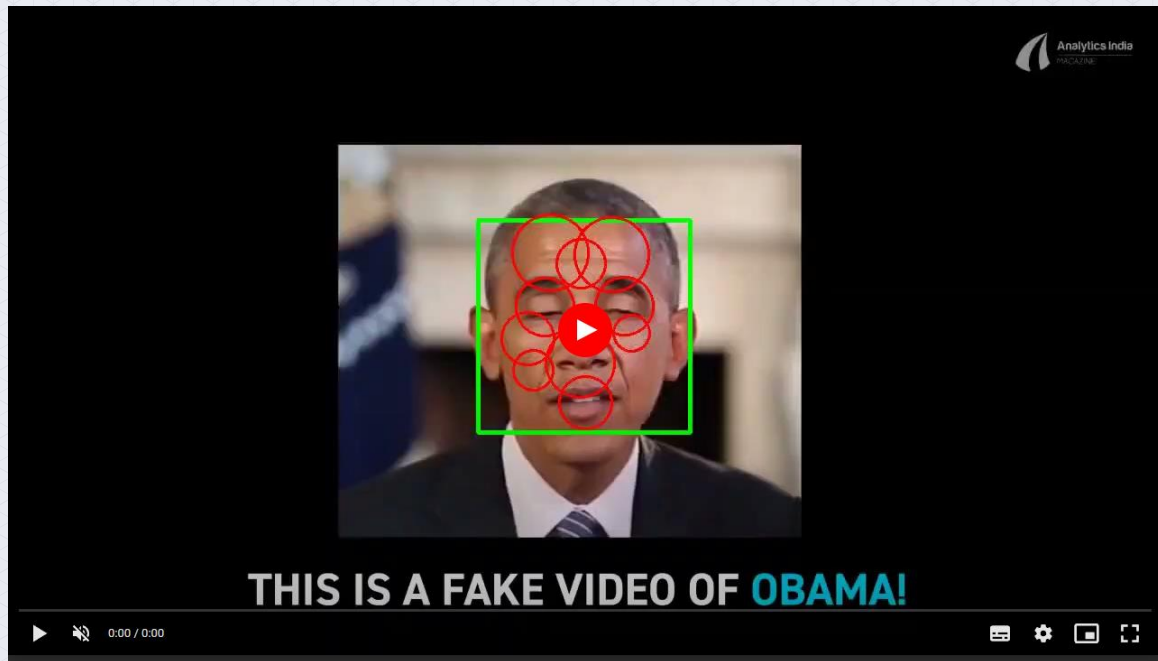
🔄 CV/Face Detection/haarcascade/video/output_obama_video.mp4



더블 클릭하여 실행

- 영상을 확인해 보고 **파라미터(parameter)**를 **조정하면서 정확도를 개선해 보자**

🔄 CV/Face Detection/haarcascade/video/output_obama_video.mp4



- 원본 동영상을 프레임 단위로 읽어서 정면 얼굴, 눈, 웃음을 인식하여 저장
- 여기서는 “rara.mp4” 원본 동영상 파일을 이용
- 얼굴(정면 얼굴, 눈, 웃음) 검출 및 저장에는 detectAndDisplay() 함수 이용
- 얼굴 검출된 출력은 “output_rara_video.mp4” 파일에 저장됨

```
1  # rara.mp4 원본 동영상에서 video stream을 읽어온다.
2
3  file_name4 = 'gdrive/My Drive/CV/Face Detection/data/video/rara.mp4' # 원본 동영상
4  output_name = 'output_rara_video.mp4' # detection 된 output 동영상
5
6  cap = cv2.VideoCapture(file_name4) # 원본 동영상
7
8  writer = None
9  if not cap.isOpened():
10     print('--(!)Error opening video capture')
11     exit(0)
12
13  while True:
14     ret, frame = cap.read()
15     if frame is None:
16         # close the video file pointers
17         cap.release()
18         # close the writer point
19         writer.release()
20         print('--(!) No captured frame -- Break!')
21         break
22     detectAndDisplay(frame) # frame 을 보낸다.
```


- VM에 저장된 “output_rara_video.mp4” 파일이 생성된 것을 확인

아래와 같이 스크립트 실행

```
1 # 구글 드라이브에 output_rara_video.mp4 파일 확인
2
3 !ls
```

```
gdrive output_rara_video.mp4 sample_data
```

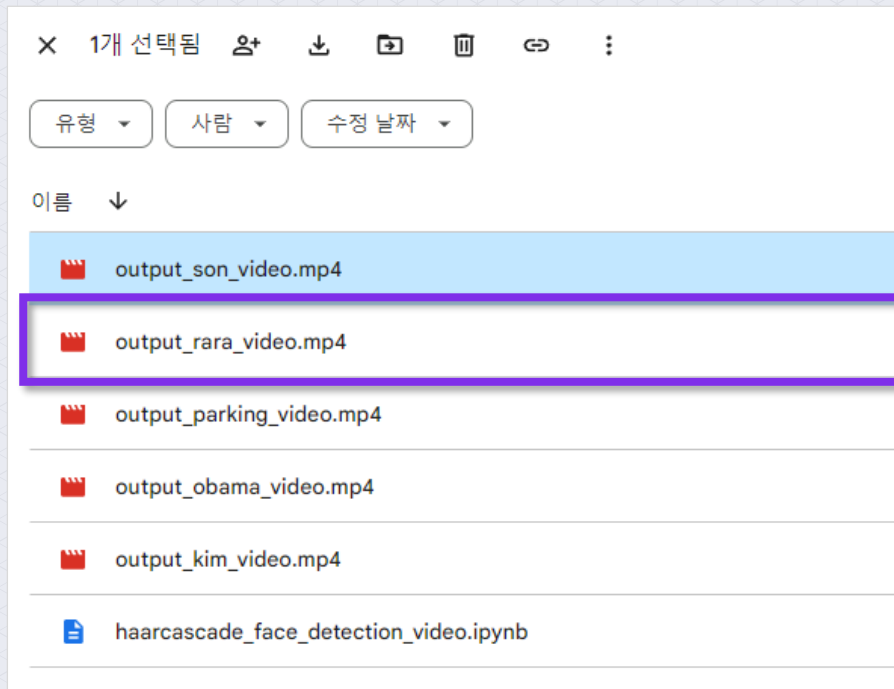
- VM에 저장된 “output_rara_video.mp4” 파일을 내 구글 드라이브로 이동시킴

아래와 같이 스크립트 실행

```
1 # 동영상에서 얼굴 검출을 저장하고, 그 파일을 내 구글 드라이브로 이동시킴
2
3 !mv output_rara_video.mp4 /content/gdrive/My Drive/CV/Face Detection/haarcascade/video/
```

■ 내 구글 드라이브에 “output_rara_video.mp4” 파일을 실행해 보자

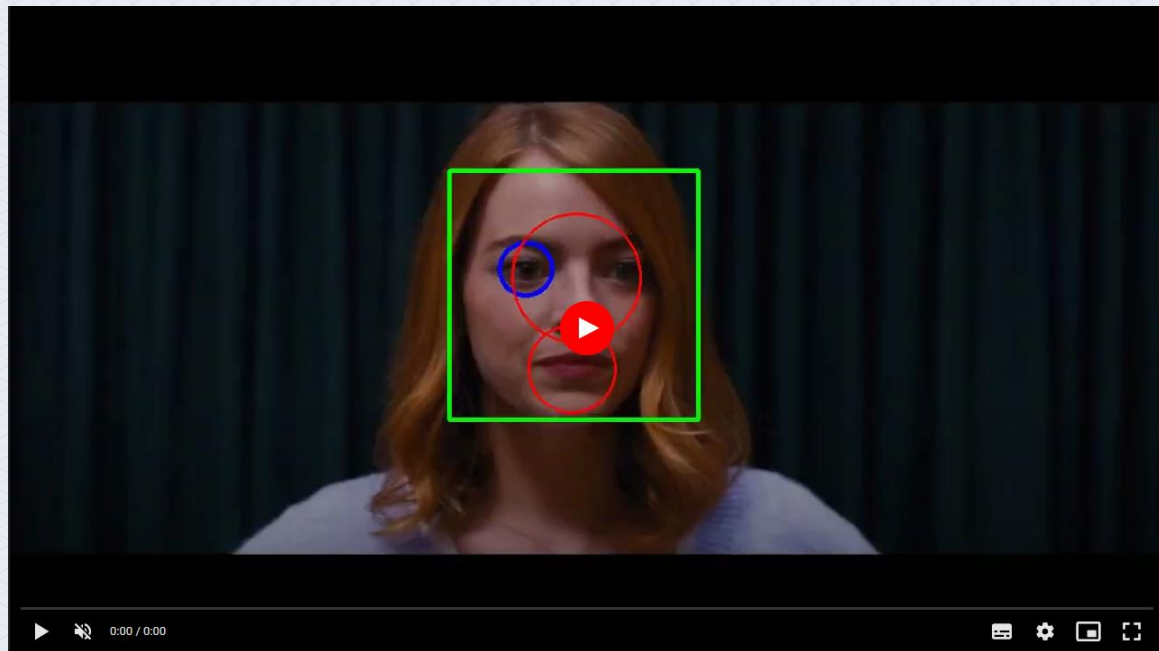
🔄 CV/Face Detection/haarcascade/video/output_rara_video.mp4



더블 클릭하여 실행

■ 영상을 확인해 보고 **파라미터(parameter)**를 **조정하면서 정확도를 개선해 보자**

🔄 CV/Face Detection/haarcascade/video/output_rara_video.mp4



- 비디오 이미지에서 차량번호판을 인식하여 정의된 파일로 저장하는 함수를 정의
- automobileDetectAndDisplay() 함수는 차량번호판을 인식하여 파일(output_name)로 저장

```

1  # automobileDetectAndDisplay() 함수는 차량번호판을 인식하고 Display하는 함수이다.
2  # - 차량번호판 인식
3
4  def automobileDetectAndDisplay(frame):
5      # GRAY 이미지로 변환
6      frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
7      frame_gray = cv2.equalizeHist(frame_gray)
8
9      # 차량번호판 인식
10     detections = plate_number_cascade.detectMultiScale(frame_gray, scaleFactor=1.1, minNeighbors=4)
11     for (x,y,w,h) in detections:
12         cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
13
14     # video 를 disk 에 output 하기 위해 writer 를 초기화한다.
15     global writer
16     if writer is None and output_name is not None:
17         fourcc = cv2.VideoWriter_fourcc(*"MJPG") # 코덱 정의
18         writer = cv2.VideoWriter(output_name, fourcc, 30, (frame.shape[1], frame.shape[0]), True)
19
20     # disk 에 frame 을 write 합니다.
21     if writer is not None:
22         writer.write(frame)
    
```

- 원본 동영상을 프레임 단위로 읽어서 차량번호판을 인식하여 저장
- 여기서는 “parking.mp4” 원본 동영상 파일을 이용
- 차량번호판 인식 및 저장에는 `automobileDetectAndDisplay()` 함수 이용
- 인식된 결과는 “output_parking_video.mp4” 파일에 저장됨

```
1  # rara.mp4 원본 동영상에서 video stream을 읽어온다.
2
3  file_name5 = 'gdrive/My Drive/CV/Face Detection/data/video/parking.mp4' # 원본 동영상
4  output_name = 'output_parking_video.mp4' # detection 된 output 동영상
5
6  cap = cv2.VideoCapture(file_name5) # 원본 동영상
7
8  writer = None
9  if not cap.isOpened():
10     print('--(!)Error opening video capture')
11     exit(0)
12
13  while True:
14     ret, frame = cap.read()
15     if frame is None:
16         # close the video file pointers
17         cap.release()
18         # close the writer point
19         writer.release()
20         print('--(!) No captured frame -- Break!')
21         break
22     automobileDetectAndDisplay(frame) # frame 을 보낸다.
```

- VM에 저장된 “output_parking_video.mp4” 파일이 생성된 것을 확인

아래와 같이 스크립트 실행

```
1 # 구글 드라이브에 output_parking_video.mp4 파일 확인
2
3 !ls

gdrive output_parking_video.mp4 sample_data
```

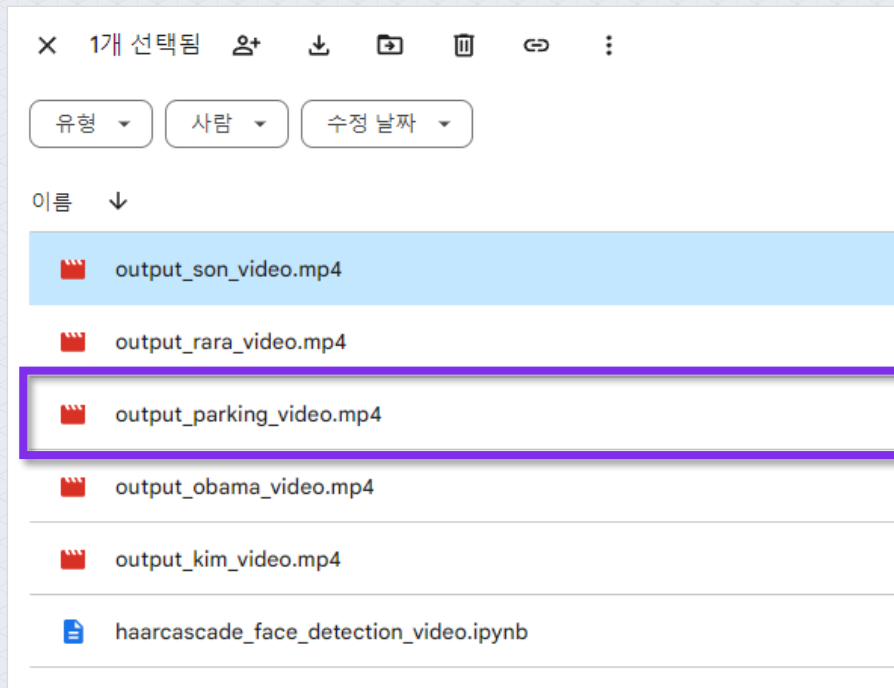
- VM에 저장된 “output_parking_video.mp4” 파일을 내 구글 드라이브로 이동시킴

아래와 같이 스크립트 실행

```
1 # 동영상에서 얼굴 검출을 저장하고, 그 파일을 내 구글 드라이브로 이동시킴
2
3 !mv output_parking_video.mp4 /content/gdrive/My Drive/CV/Face Detection/haarcascade/video/
```

■ 내 구글 드라이브에 “output_parking_video.mp4” 파일을 실행해 보자

🔄 CV/Face Detection/haarcascade/video/output_parking_video.mp4



더블 클릭하여 실행

- 영상을 확인해 보고 **파라미터(parameter)**를 **조정하면서 정확도를 개선해 보자**

🔄 CV/Face Detection/haarcascade/video/output_parking_video.mp4

